Signal Processing: Image Communication 130 (2025) 117173

Contents lists available at ScienceDirect

Signal Processing: Image Communication

journal homepage: www.elsevier.com/locate/image

Distributed virtual selective-forwarding units and SDN-assisted edge computing for optimization of multi-party WebRTC videoconferencing

R. Arda Kırmızıoğlu [a,*], A. Murat Tekalp [b], Burak Görkemli [c]

[a] TomTom, Amsterdam, The Netherlands
[b] Department of Electrical and Electronics Engineering, Koç University, Istanbul, Turkey
[c] ARGELA Technologies, İzmir, Turkey

ARTICLE INFO

Keywords:
Multi-party webRTC
Selective forwarding unit (SFU)
Edge networks
Distributed virtual SFU
Managed video service

ABSTRACT

Network service providers (NSP) have growing interest in placing network intelligence and services at network edges by deploying software-defined network (SDN) and network function virtualization infrastructure. In multi-party WebRTC videoconferencing using scalable video coding, a selective forwarding unit (SFU) provides connectivity between peers with heterogeneous bandwidth and terminals. An important question is where in the network to place the SFU service in order to minimize end-to-end delay between all pairs of peers. Clearly, there is no single optimal place for a cloud SFU for all possible peer locations. We propose placing virtual SFUs at network edges leveraging NSP edge datacenters to optimize end-to-end delay and usage of overall network resources. The main advantage of the distributed edge-SFU framework is that each peer video stream travels the shortest path to reach other peers similar to mesh connection model, whereas each peer uploads a single stream to its edge-SFU avoiding the upload bottleneck. While the proposed distributed edge-SFU framework applies to both best-effort and managed service models, this paper proposes a premium managed, edge-integrated multi-party WebRTC service architecture with bandwidth and delay guarantees within access networks by SDN-assisted slicing of edge networks. The performance of the proposed distributed edge-SFU service architecture is demonstrated by means of experimental results.

1. Introduction

Recent advances in software-defined networking (SDN), network function virtualization (NFV), and data center infrastructure have led to two major industry trends: (i) cloud service providers are moving towards network edges for lower latency and higher bandwidth access (also known as fog computing), (ii) network service providers (NSP) are replacing closed and proprietary hardware-based access technologies with disaggregated and virtualized software running on edge clouds to place network intelligence and services at network edges [1]. Federated edge clouds supporting SDN-assisted network virtualization and slicing play a key role in operating 5G networks to unlock the full potential of new and emerging services. Real-time communications (RTC) services will be a major beneficiary of this trend.

WebRTC [2] is a prominent open framework for RTC that provides browser-to-browser voice, video, and data communications via simple Javascript APIs. VP9 [3] is a royalty-free codec that allows for efficient scalable video coding. WebRTC and VP9 are supported by leading browsers such as Firefox and Chrome. WebRTC with VP9 video encoding is currently available over *the best-effort* Internet. In RTC services, media communication between multiple parties is facilitated through a middle-box called selective-forwarding unit (SFU). Providing a desired level of quality of experience in multi-party RTC videoconferencing/telepresence requires connections with low latency and guaranteed bandwidth. However, this is not always possible using a middle-box anywhere over the best-effort Internet, and end-to-end delay and video quality often exhibit undesirable fluctuations due to sudden network state variations even though WebRTC employs powerful network estimation and rate adaptive video encoding schemes. Hence, it is important to manage video RTC services at network edges to provide low latency and guaranteed bandwidth.

This paper proposes two innovations that significantly improve the quality of multi-party WebRTC services leveraging SDN-assisted NSP edge datacenters: (i) We propose to deploy virtual SFUs at network edges (edge datacenters), where each edge-SFU processes the upstream originating from peers in that access network, and (ii) While the proposed edge-SFU framework applies to both best-effort and managed service models, this paper proposes an edge-integrated managed service by provisioning resource-reserved slices to offer low-delay,

* Corresponding author.
E-mail address: rizaardakirmizioglu@gmail.com (R.A. Kırmızıoğlu).

https://doi.org/10.1016/j.image.2024.117173
Received 7 February 2022; Received in revised form 27 January 2023; Accepted 15 July 2024
Available online 12 September 2024
0923-5965/© 2024 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

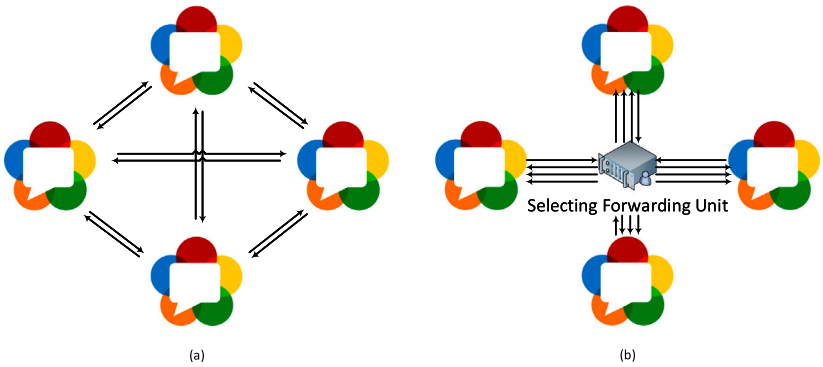


Fig. 1. Multi-party real-time communications service topologies: (a) Mesh connection topology, (b) MCU/SFU connection topology.

guaranteed quality of service within access network domains. The proposed managed multi-party WebRTC service features the following advantages:

- minimum end-to-end delay between all pairs of parties in a multi-party WebRTC session
- minimum overall core network bandwidth consumption
- guaranteed bandwidth to all parties in a multi-party WebRTC session within edge access networks

While the first and last features benefit end users, the second one benefits the NSP. The guaranteed bandwidth within edge networks provides each party predictable and consistent video quality throughout a session. We assume that bandwidth is largely overprovisioned in the core network; hence, no bandwidth reservation is needed within the core network.

The rest of the paper is organized as follows: Section 2 reviews basics of multi-party WebRTC, scalable video coding, and edge datacenters. Related work and novelty of the paper are discussed in Section 3. The proposed edge-integrated low-delay WebRTC service architecture is introduced in Section 4. Section 5 describes providing premium guaranteed bandwidth real-time communications services over resource-reserved slices in edge-access networks. Experimental results are presented in Section 6. Finally, Section 7 presents conclusions.

2. Background

2.1. Multi-party WebRTC service models

WebRTC consists of an IETF protocol specification and Javascript APIs defined in a series of W3C documents [4]. WebRTC APIs allow media capture, and sharing media and data streams between browsers. WebRTC uses Session Traversal Utilities for NAT (STUN) servers to help end-points find each other in the presence of network address translation (NAT), which maps a group of private IP addresses to a single public IP address, and use Traversal Using Relays around NAT (TURN) servers as relays in the presence of firewalls.

A signaling server establishes connections between peers and creates media channels. Signaling methods and protocols are not specified by WebRTC. WebRTC media streams are delivered point-to-point using the real-time transport protocol (RTP) over the user datagram protocol (UDP). Real-time transport control protocol (RTCP) is used to monitor and estimate the available bandwidth between the end points. WebRTC platform [2] employs the Google congestion control algorithm [5] for rate control of the VP9 video encoder. Recently, the WebRTC specification has been extended to enable configuration of encoding parameters for Scalable Video Coding (SVC) [6].

WebRTC sessions between two peers are point-to-point, while multi-party sessions can be mesh connected or connected through a central router, which are discussed in more detail in Sections 2.1.1 and 2.1.2, respectively.

2.1.1. Mesh connection model

In the mesh connection model, each peer establishes a direct point-to-point connection with all other peers as depicted in Fig. 1(a). Mesh topology is advantageous in terms of low delay, since direct connections require the smallest number of total hops between endpoints. However, it uses the limited upload bandwidth of peers inefficiently, since each peer needs to send duplicate copies of the video and audio upstream to each other peer. In the default mesh connection scenario, each peer runs a single non-scalable video encoder in order to avoid the computational burden of running multiple encoders simultaneously. Hence, the video encoding rate of each peer is determined according to the minimum of its own upload bandwidth and the download bandwidths of all receiving peers. Due to the asymmetric nature of broadband access, the bottleneck is typically the peer upload bandwidth, which makes mesh connection model undesirable.

2.1.2. Middle-box connection model

Each peer sends a single video stream to a central middle-box router, as shown in Fig. 1(b), which forwards them to other peers according to session configuration. This model avoids the upload bandwidth bottleneck at peers.

If the middle-box performs transcoding and/or mixing of incoming streams to accommodate peers with different bandwidths and video resolution capability, it is called a multi-point control unit (MCU).

Selective forwarding unit (SFU) is a simpler router/switch that selectively forwards layers of scalable video to peers according to session configuration, where each peer runs a scalable codec and sends all layers of encoded video to the SFU. Unlike the MCU, the SFU introduces no processing delay, since it only forwards packets without processing the payload. In its standard usage, it is a single point of failure, since all peers must send streams to the same SFU.

2.2. Scalable video coding

There exist several scalable video coding standards, including H264/AVC [7], H265/HEVC [8], VP9 [9] and AV1 [10]. We selected scalable VP9 coding, which is an open and royalty-free codec that outperforms H.264/AVC in terms of rate–distortion performance at the expense of slightly higher complexity [11]. Yet, it is possible to perform real-time VP9 encoding/decoding at 50 Hz on a standard laptop for high definition (HD) video using libvpx codec implementation [3].

VP9 offers support for mixed temporal and spatial scalable video coding; that is, it produces a hierarchy of spatial and temporal layers within the same encoded bit stream. In the flexible payload format mode, layer hierarchy and dependencies can be changed dynamically. In the non-flexible mode, the layer hierarchy and dependencies within the group of frames (GOF) are pre-specified by the scalability structure data. Fig. 2 depicts a scalable video stream, with 2 spatial and 3 temporal layers, where arrows show the dependency structure of layers. Layers that do not depend on other layers can be dropped without

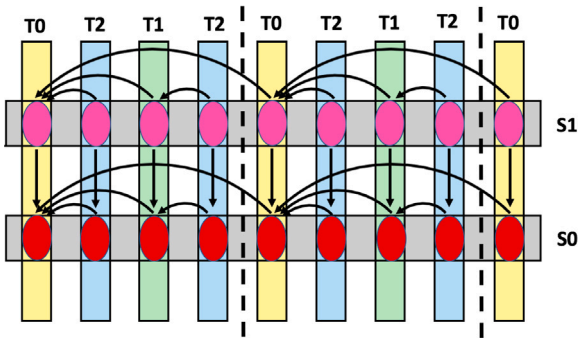


Fig. 2. Mixed spatio-temporal scalable coding with two spatial (S0, S1) and three temporal (T0, T1, T2) layers and the dependency between the layers.

affecting decodability of the received bitstream. Hence, coarse-grain rate control can be achieved by discarding one or more spatial and/or temporal layers per frame at the SFU. Finer bitrate control can be achieved by adjusting the quantization parameter while encoding at peers [12].

2.3. Multi-access edge datacenters and network slicing

Access networks, including residential, business, and mobile networks, are evolving into edge clouds managed by distributed edge datacenters, realized by using white-box hardware and open source software, leveraging SDN and NFV technologies featuring virtualization capability. We use "edge datacenter" as a generic term that refers to the Central Office Re-architected as a Datacenter (CORD) architecture [13] that is a cloud-native open platform proposed by Open Networking Foundation (ONF) (or similar platforms proposed by other standards groups) to help NSPs in this edge cloud evolution. The SDN-Enabled Broadband Access (SEBA) [14] is an exemplar CORD platform that is built by the ONF community to transform management of Passive Optical Network (PON) based fixed broadband access network domains.

Each edge datacenter runs an SDN controller (SDN-C) that communicates with switches in the access network using OpenFlow through its south-bound interface, and with controller applications such as the traffic engineering manager (TEM) using the REST API through its north-bound interface. Each access network domain is connected to the core network by an aggregation switch that is a gateway between local switches within the access network domain and the core network. We propose that each multi-access edge datacenter also hosts video service functions, such as a virtual SFU application.

Although video traffic is increasing significantly every year, this is not contributing to the revenues of the NSPs. With SDN-assisted edge datacenters, NSPs can increase their revenues by introducing innovative services that require low latency and guaranteed bandwidth. Network slicing provides the ability to partition the network into virtual pipes with specific latency and/or bandwidth guarantees, that could change the underlying economics of network services. Today, managed uni-directional video streaming services, such as IPTV solutions, are offered over closed networks. The advent of NSPs deploying SDN-assisted edge clouds provides an important opportunity for NSPs or third-party service providers to offer premium uni-directional streaming or interactive RTC video services with quality-of-service over open networks to increase their revenues.

3. Related work and novelty

Recently, solutions for low latency live streaming, where the goal is to achieve an end-to-end latency in a range of three to five seconds when delivering live content from a live source to multiple viewers were proposed [15,16]. However, interactive RTC applications that we consider in this paper have much stricter delay requirements, typically less than 150 ms.

There exist a limited number of published works on multi-party WebRTC videoconferencing with scalable video coding as a cloud service and also very few on managed WebRTC services over SDN, which are discussed below.

A proprietary multi-party videoconferencing system using SVC and a central server was first proposed in [17] to avoid transcoding delays at the MCU. Later, the term selective forwarding unit (SFU) was coined for the server middlebox. With the emergence of WebRTC framework, multi-party WebRTC videoconferencing using VP9 SVC and an open source SFU implementation that forwards layer streams received from sending peers according to capabilities of receiving peers has become a de facto standard. A comparison of open source WebRTC SFU implementations is presented in [18]. Other related works include a cloud-based implementation of the MCU/SFU [19] and a peer-to-peer (P2P)-MCU architecture, where MCU functionality is hosted in one of the peers [20].

Launching WebRTC services in SDN environment was discussed in a concept paper on "network as a service" [21], which did not present an implementation. Realizing dynamic-network-enabled RTC on a proof-of-concept 5G network was discussed in [22]. However, none considers a distributed edge-SFU implementation for peers in a single session. Among our previous works on SDN-assisted multi-party WebRTC services, [23] considers rate adaptation and quality of service provisioning for mesh-connected clients and [24] proposes an SDN-assisted IP-multicast scheme, where OpenFlow (OF) switches in the network can be configured to split an incoming stream from a send-only PeerConnection object to be forwarded to multiple receive-only PeerConnection objects. The latter presents a method to find optimal OF switches anywhere in the network to split an upstream to serve multiple receiving peers. There are no SFUs and no distinction has been made between access networks and core network in [24], since the upstream can be split at any OF switch guided by the SDN controller. Furthermore, although provisioning uni-directional HTTP-adaptive video streaming services over reserved network slices has been previously proposed [25,26], leveraging edge clouds and reserved network slices to improve performance of interactive real-time communication services has not been studied.

To the best of our knowledge, there is no published work on architectures that leverage SDN-assisted NSP edge datacenters in order to improve the performance of a WebRTC service. This paper proposes a solution to fill this gap. Our novel contributions are:

- We propose a distributed edge-SFU service architecture, whereby a single WebRTC session can be served by more than one edge-SFU, which helps minimize end-to-end delay and overall network resource usage.
- The proposed distributed edge-SFU framework combines the best aspects of mesh connection and classic single SFU connection models in that each peer video stream travels the shortest path (hence low-delay and core network resource savings) similar to the mesh connection model, while each peer uploads a single video stream to its edge-SFU avoiding the upload bottleneck problem.
- The proposed distributed edge-SFU framework is robust to SFU overload/congestion or failure since if one of the edge-SFUs fail, the WSS can assign one of the other edge-SFUs to peers in a given access network to temporarily replace the failed SFU.
- The proposed edge-SFU framework allows operators to offer value-added managed WebRTC services over reserved network slices at network edges to provide guaranteed bandwidth over edge access networks.
- We propose to integrate virtual edge-SFUs into the emerging SDN-assisted NSP edge datacenters so that offering the proposed edge-SFU service becomes feasible without any additional infrastructure investment cost.

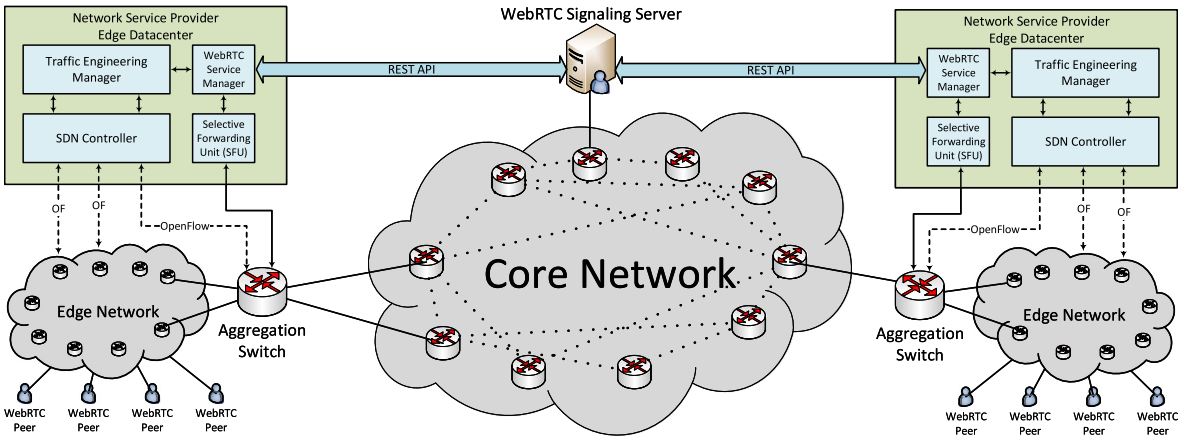


Fig. 3. NSP topology with the core network and two access domains managed by multi-access edge datacenters hosting edge-integrated WebRTC service.

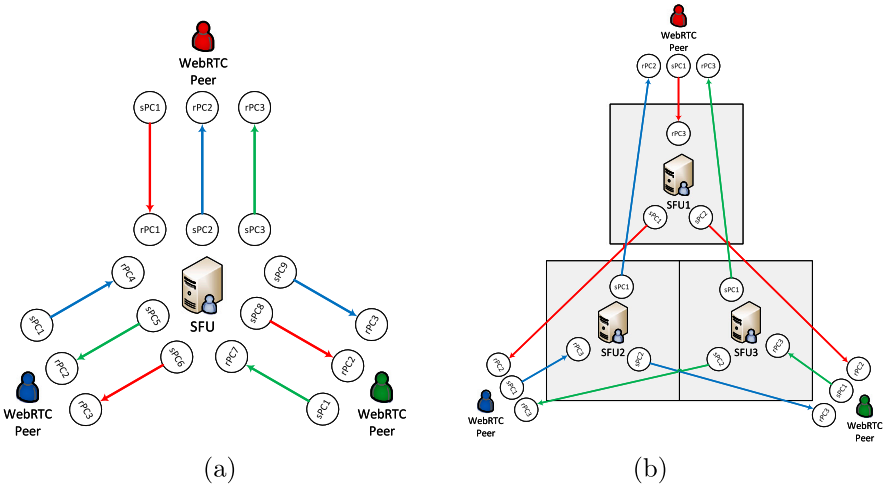


Fig. 4. SFU service logical system architectures: (a) Single SFU service architecture, (b) Proposed distributed edge-SFU architecture.

4. Edge-centric WebRTC service architectures

The main design principle of WebRTC is to generate matching PeerConnection object (PCO) pairs to send and receive data via point-to-point communication between pairs of endpoints, that is, each receive PCO (rPCO) is connected to only one send PCO (sPCO). In a mesh-connected N-party session, each peer creates $N-1$ sPCO and $N-1$ rPCO. As a result, the upload bandwidth of peers becomes a bottleneck in a mesh-connected session. In an SFU-connected session, each peer creates just one sPCO (to the SFU) and $N-1$ rPCO, while the SFU server generates N rPCO to receive streams from each peer, and $N(N-1)$ sPCOs to duplicate each received stream $N-1$ times and forward them to each receiving peer through a different sPCO.

In the state-of-the-art multi-party WebRTC services using scalable video coding, all peers in a session are served by a single SFU, implemented as a cloud service. The location of the cloud SFU in the network plays an important role in the end-to-end delay performance of the session. The options for placing the SFU in this traditional service model are discussed in Section 4.1.

With the emerging paradigm of managing access networks as edge clouds, where the NSP network consists of a core network and a number of edge access network domains, NSPs can provide virtual SFU (vSFU) services at every edge cloud. The overall topology of edge-cloud based access networks, including the proposed virtual edge-SFU service provided by the NSP at edge clouds, is depicted in Fig. 3. In Section 4.2, we propose a novel distributed edge-SFU framework, where a vSFU is implemented in every edge cloud, and each peer uses the vSFU in its edge domain for its upstream data.

4.1. Single SFU service

In the traditional single SFU service, all peers in a session send and receive video streams from the same central SFU server. If there are N peers in a session, then each peer creates one sPCO and $N-1$ rPCO. The SFU, on the other hand, creates N rPCO and $N(N-1)$ sPCO (N^2 PCO in total) to serve all peers. The traditional single SFU service architecture with 3 peers in a session is illustrated in Fig. 4(a), where streams originating from each peer are shown in different colors. The location of the SFU switch affects the end-to-end (E2E) delay experienced by the peers in a session as well as the overall core network bandwidth consumption. We consider two possibilities for the location of the SFU for a given session to demonstrate these points in our simulations:

4.1.1. Cloud-SFU service

This is the most common implementation of the traditional SFU streaming service. A cloud SFU switch can be placed anywhere in the global network. In our simulations (for performance comparison purposes) this location is selected at random.

4.1.2. Single edge-SFU service

A virtual SFU (vSFU) switch is placed in each edge cloud as depicted in Fig. 3, and all peers in the same session are served by the same vSFU

located in the edge cloud of one of the peers (randomly selected in our simulations) in the session.

4.2. Proposed distributed edge-SFU service

In the proposed distributed edge-SFU architecture, where a single session is served by multiple vSFUs, a vSFU shall be placed in each edge cloud and each peer connects to the vSFU located in its own edge network. The proposed distributed SFU service architecture is illustrated in Fig. 4(b) for a session with 3 peers that are in different edge domains, where video stream from each peer is shown by a different color.

A WebRTC session with the distributed SFU architecture can be instantiated by pairing rPCO of each peer to the sPCO of vSFU of the respective sending peer and vice versa. Note that the total number of PCO created by all vSFUs serving a session with N peers is still N^2.

4.3. Comparison of single vs. Distributed edge-SFU service

The proposed distributed edge-SFU service features near-shortest-path service between peers similar to the mesh architecture depicted in Fig. 1(a) without the upload bandwidth bottleneck of mesh service. Experimental results indeed validate that the distributed edge-SFU service architecture has lower E2E delay between peers and lower NSP overall network resource consumption compared to the traditional single SFU service.

5. WebRTC services over edge network slices

A traditional WebRTC service is a best effort service with no control over network resources. This section introduces a new value-added real-time communications (RTC) service framework for delivering WebRTC streams with assured quality of service within edge networks. This service can be provided by the NSP itself or by a third-party video service provider (VSP) that collaborates with the NSP. The proposed value-added service framework is introduced in Section 5.1. Dynamic reservation of edge network slices and implementation of reserved slices by queue management at edge network switches are discussed in Section 5.2.

5.1. Value-added RTC service orchestration at network edges

In the proposed RTC service model, the compute node of each NSP edge cloud hosts a vSFU and a WebRTC service manager (WSM) application to serve peers in that edge domain. All peers in an edge domain send their upstream to the vSFU in their domain as shown in Fig. 4. The NSP offers per-flow quality of service (QoS) for both upstream and downstream WebRTC flows within each edge domain by providing minimum delay network slices with specified bandwidth via the WSM of each edge domain. The WebRTC service orchestration between the peers, domain WSMs, and edge vSFUs for each session is realized by the *Signaling Server* (SS), which is a central cloud service. The central WSS sends session information to all peers, as well as vSFU and WSM of each edge domain in a session. The functionality of the central WSS and local WSM applications are described in more detail below:

5.1.1. WebRTC Signaling Server (WSS)

Peers establish a session by exchanging Session Description Protocol (SDP) [27] and Internet Connectivity Establishment (ICE) [28] objects through a central WSS. ICE objects contain IP address, transport protocol and port number of the peers. The WSS also creates data sockets for each peer. WSS keeps a dictionary with peer IP-addresses and their matching edge domain vSFU IP-addresses as key–value pairs so that WSS knows which vSFU each peer is connected to. There exists two-way communication between the central WSS and WSM of each edge domain. In addition to passing peer information to the WSM, the SS receives slice reservation confirmation from the WSM and passes them back to peers.

5.1.2. WebRTC Service Manager (WSM)

WSM application in each edge domain gathers session information about peers from the WSS in order to create peer instances. The WSM manages various services that the NSP provides to WebRTC peers, such as slice reservation, queue management, providing network statistics, and policing. WSM also communicates slice reservations and other session information back to the WSS.

WSM application passes peer instance information to the Traffic Engineering Manager (TEM) of the SDN controller in its edge domain, which then (re)computes a path (network slice) between peers (as described in Section 5.2) and defines/updates flow tables at the switches accordingly. TEM also periodically allocates, updates, or destroys queues at the switches, and assigns WebRTC flows to the allocated queues. Details of these steps are discussed in Section 5.2. Finally, TEM passes per-flow path reservation and other flow information back to the WSM app, which sends them to peers through the WSS so that peers can select appropriate encoded video layers to be sent over these network slices.

5.2. Dynamic slice management at edge networks

In order to support value-added RTC services, the NSP provides bandwidth and delay constrained network slices between desired end-points specified by the WSM app. Network slice reservation within each edge domain and implementation of reserved slices at switches are carried out by the TEM unit under the supervision of the SDN controller as follows:

5.2.1. End-to-end slice request and resource reservation

WSM application at each edge domain classifies all flows requested to be set up as internal or external flows by analyzing their origin and destination IP addresses and passes resource reservation requests to the *Traffic Engineering Manager* (TEM), which is a SDN controller application. TEM reserves path and bandwidth for both send and receive video streams in a session depending on whether they are internal or external flow as follows:

Internal Flows: When the $sender_i$ and the $receiver_i$ are in the same edge network, TEM runs Algorithm 1 to find a bandwidth constrained, minimum delay path between these end points. Then, the SDN controller allocates priority queues matching the flow along the computed path.

External Flows: The TEM runs Algorithm 1 between the aggregation switch and the end-point that can be either SFU or peer. Then, at each edge, the related SDN controller allocates priority queues matching the flow along the computed path.

5.2.2. Constrained shortest-delay path finder algorithm

Since the SDN controller of each edge domain has full visibility of switch queue states and link traffic in its network, the TEM can search for the minimum delay path that satisfies the desired bandwidth requirement, and the NSP can route premium WebRTC traffic over these reserved paths. We employ the bandwidth-constrained shortest-delay path finder (CSPF) algorithm, summarized in Algorithm 1, for path computation between endpoints in both directions, since the uplink and downlink paths between endpoints need not be the same.

The inputs to Algorithm 1 are a directed network graph G that consists of vertices (V) and edges (E), and the desired bandwidth bw_i from the source peer $sender_i$ to destination peer $receiver_i$. The outputs are the optimal path p_i^* and the minimum delay $delay_i$ over the path p_i^* with hop count hc_i.

The CSPF algorithm performs a sub graph creation G^*, trimming the links with available bandwidth below the desired bandwidth, so that each link in G^* has at least bandwidth bw_i available. Then, it runs Dijkstra's Shortest Path algorithm [29], where link delays are considered as cost and finds the path that satisfies required bandwidth value bw_i with the minimum delay $delay_{p^*}$.

Algorithm 1: Constrained Shortest-Delay Path Finder

Input: G, $sender_i$, $receiver_i$, bw_i
Output: p_i^*, $delay_i$, hc_i
$p_i^* \leftarrow \emptyset$ (Set of links)
$delay_{p^*} \leftarrow$ Null
$hc_i = 0$
Create graph G^* such that $\forall$Link $\in G^* \geq bw_i$
$p_i^* \leftarrow$ Dijkstra(G^*, src_i, dst_i)
$delay_{p^*}$ = Sum of all latency delay over p_i^*
hc_i = Number of links over p_i^*

Complexity Analysis: The complexity of creating the graph G^* with adjacency list data structure is $O(|E| + |V|)$ and running the Dijkstra's Shortest Path algorithm [29] with binary heap is $O(|E|log|V|)$. Therefore, the worst-case complexity of Algorithm 1 is $O\big(|V| + |E|log|V|\big)$.

5.2.3. Implementation of reserved slices in edge networks

Once end-to-end paths with the desired QoS parameters are calculated, the resulting network slice reservation is implemented at the switches by means of queue management tools.

6. Evaluation

This section first describes a small-scale proof-of-concept emulation environment to show feasibility of implementation of the proposed architecture with real components and protocols in Section 6.1. We then discuss feasibility of integration of the proposed service with a SEBA edge datacenter in Section 6.2. Next, we present experimental results to evaluate the performance of proposed edge-based distributed-SFU RTC service architecture on a large-scale simulation environment. The simulation environment and test conditions are presented in Section 6.3. Experimental results to evaluate E2E delay and overall network resource consumption for different SFU configurations are provided in Section 6.4.

6.1. Proof-of-concept emulation

We first built a proof of concept system in a laboratory setting in order to demonstrate that the proposed service architecture can be successfully deployed over real network protocols. To this effect, we emulated three separate SDN-enabled access networks using different Mininet [30] instances each with 10 Open vSwitch instances. Each access network is controlled by a different Floodlight SDN controller [31] and is connected to the core network via a single border switch. A Janus SFU [32] instance is connected to the border switch of each access network. There is a single host in each access network running a native WebRTC peer application that supports scalable VP9 video codec. We run each network emulation in a different PC to achieve real-time video encoding/decoding performance at virtual hosts, where different Mininet instances are connected by GRE tunnels. A WebRTC Signaling Server (WSS) is placed at a random location in the core network (as its location has no effect on the operation of the system). The core network does not have to be SDN-enabled and there is no controller for the core network in our implementation. The emulation setup is depicted in Fig. 5.

The WSS stores the location of each edge-SFU in a main key–value table, where the subnet IP address for each edge network is the key and IP address of the respective edge-SFU is the corresponding value. In a multi-party WebRTC session with N peers, each peer has one send-only PeerConnection object and $N-1$ receive-only PeerConnection objects. A separate session table is kept for each WebRTC session to store PeerConnectionObject-SFU relations. Whenever a peer logs in WSS to start or join a WebRTC session, the subnet IP addresses of peers in the session are used as keys to retrieve from the main table the IP addresses of SFUs that its send-only PeerConnection object and $N-1$ receive-only PeerConnection objects will connect. These PeerConnectionObject-SFU mappings are kept in the session table. In short, the main table is used to build the session table, which is used to assign an SFU to each PeerConnection object of each peer in a session during session start-up.

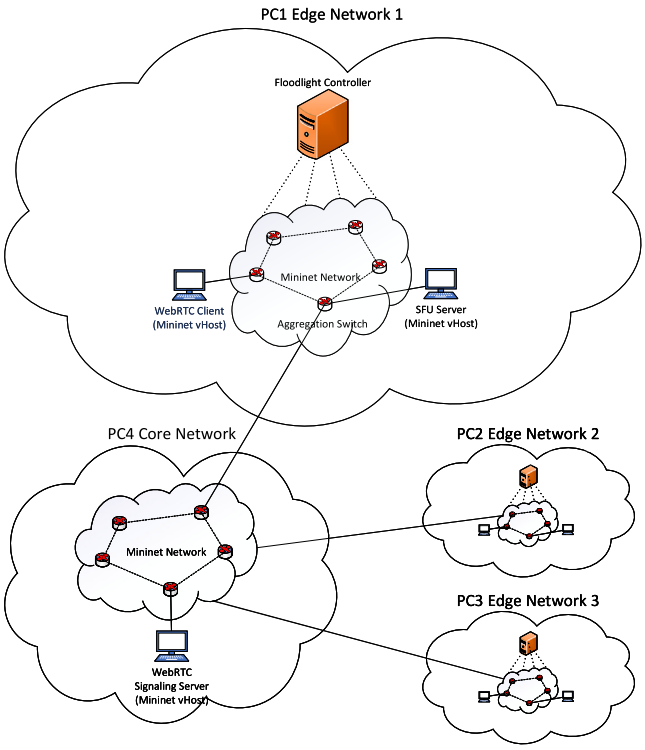


Fig. 5. A proof-of-concept emulation environment in the lab setting. Three PCs run access network emulations (together with a Janus SFU connected to the border switch of each access network and a host running WebRTC peer application) and one PC runs core network emulation with a WSS.

During session start-up, WSS communicates with the peers and their SFUs to set-up PeerConnection objects. WSS configures each peer's send-only PeerConnection object with the IP address of the SFU at its edge datacenter using the session table. In the traditional single SFU set up, WSS sets-up N^2 PeerConnection objects at the SFU (N receive-only objects to receive streams from each peer and $N(N-1)$ send-only objects to forward each received stream to other peers). In the proposed system, we modified the WSS, such that WSS now sets-up N PeerConnection objects (one receive-only and $N-1$ send only objects) in each of the N SFUs located at edge datacenters using the session table. Note that each edge-SFU processes only the stream received from the send-only PeerConnection object of the peer located in that access network to forward it to receive-only objects of other peers.

6.2. Integration into SEBA multi-access edge datacenters

The proposed service architecture can easily be integrated into the SDN-Enabled Broadband Access (SEBA) exemplar platform that is built by the ONF community. An example of such an implementation is shown in Fig. 6, where Passive Optical Network (PON) based broadband network access of WebRTC peers is managed by the SEBA platform. The SEBA Edge Manager components run on compute nodes, which are connected to the data plane over a P4-based leaf-spine fabric. The additional components that are introduced by the proposed edge-SFU solution, the SFU control plane (SFU-C) and the WebRTC Service Manager, will also run on the compute nodes of the edge datacenter,

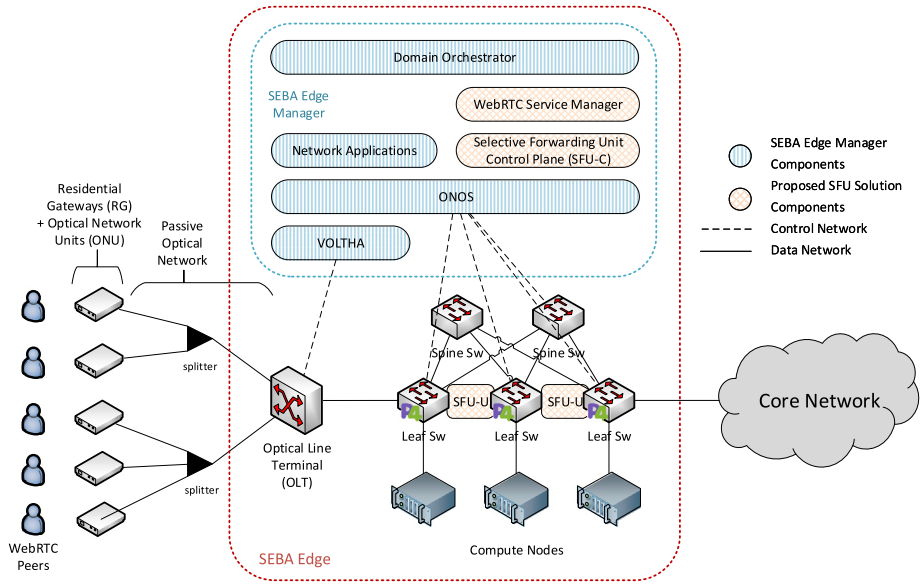


Fig. 6. Integration of the proposed edge-SFU service architecture into a SEBA edge datacenter.

whereas the P4-based SFU user plane (SFU-U) will run on the leaf switches.

The SEBA Edge Manager components are responsible for managing the PON network, presenting end users multiple services, such as High Speed Internet Access, Voice over IP, IPTV and Video on Demand, each having different QoS requirements. Virtual Optical Line Terminal Hardware Abstraction (VOLTHA) abstracts the details of the PON network, presenting a switch-based model to Open Network Operating System (ONOS), which is the SDN controller of choice for the SEBA solution. ONOS controls both PON and the leaf-spine fabric, enabling end-to-end connection between end-users and the core network. Network Applications that run on top of ONOS implement the logic to realize the traffic management and QoS requirements for different services. Domain Orchestrator manages the life-cycle of services and subscribers, as well as presenting an integration point for operators' Operations Support Systems (OSS)/Business Support Systems (BSS).

The proposed edge-SFU solution introduces the SFU function, separated into control plane (SFU-C) and user plane (SFU-U) components, and the WebRTC Service Manager to the SEBA Edge. SFU-C can be considered as a network application that runs on top of ONOS and controls SFU-U, which is implemented as a P4 program running on the leaf-spine fabric. SFU-U parses the layer identifiers of the scalable videoconference packets from RTP headers and either forwards or discards them depending on the destination peer and the flow rules dictated by SFU-C. WebRTC Service Manager communicates with the centralized WebRTC Signalling Server to manage the selective flow forwarding process through SFU-C/U.

6.3. Simulation environment

We provide results over a large scale simulation environment using a core network with thousands of switches and twenty edge domains as depicted in Fig. 7 and one hundred multi-party WebRTC sessions involving hundreds of peers.

6.3.1. NSP network simulation

We simulated core and edge networks of different sizes with random topology, where each edge network has 16, 25, 36, 49, 64, 81, 100 switches and core network has 256, 400, 576, 784, 1024, 1296, 1600 switches, respectively. Each edge network is connected to the core network with a single aggregation switch. We employ a physical distance based probabilistic model [33] to create bi-directional links

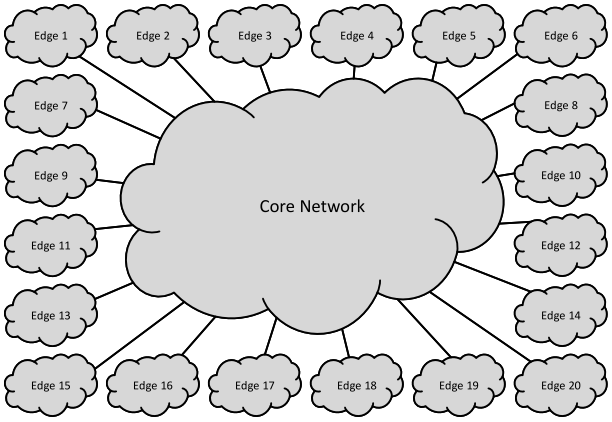


Fig. 7. Simulated network topology with 20 edge domains.

between pairs of switches in both core and edge networks. In order to model physical distances between nodes, we consider a rectangular grid of unit squares, where there are as many squares as the number of switches in both core and edge networks. Each switch is placed at a random coordinate within a unit square. The probability P_{ij} of a link between two nodes i and j is

$$P_{ij} = \beta \exp\left(\frac{-d_{ij}}{L\alpha}\right) \tag{1}$$

where d_{ij} is the Euclidean distance between the nodes, L is the maximum Euclidean distance between any two nodes, and α and β are parameters. In the simulations, we assume each hop introduces a fixed transmission delay of 5 ms, which is determined as the average of *Traceroute* measurements [34] from our server to different university servers across the globe.

6.3.2. WebRTC service simulation

Once a network topology is generated, we place peers in each WebRTC session randomly using a two-step procedure: (1) we determine an edge network for each peer in a session according to a uniform probability distribution. (2) we select a node in the edge network that the peer is connected to according to a uniform probability distribution.

In the proposed edge-based WebRTC service, each peer runs a scalable video encoder configured to encode video with two spatial

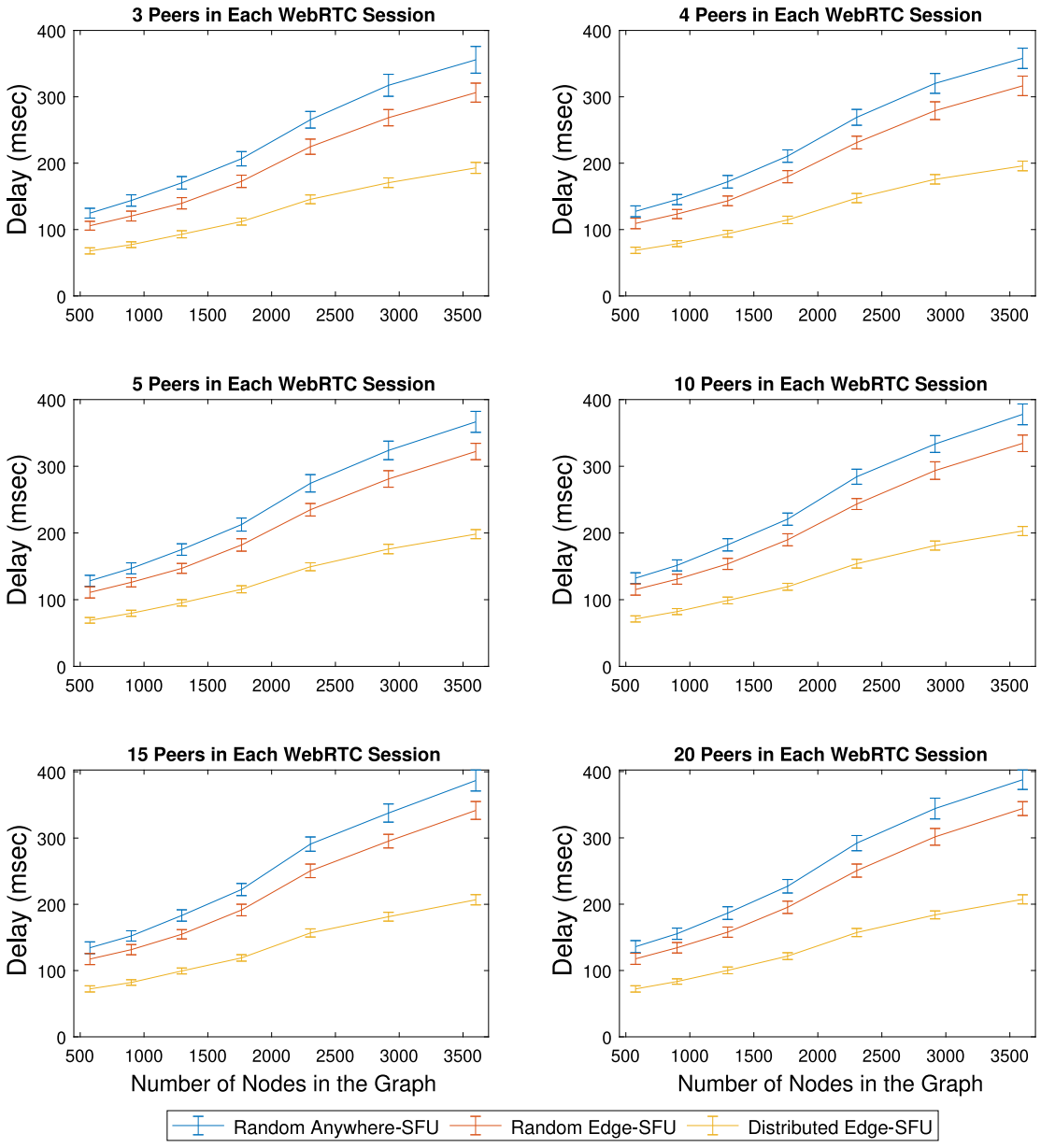


Fig. 8. Mean of the maximum E2E delay in 100 trials of 100 services. It can be seen that the maximum E2E delay in the proposed distributed edge-SFU framework is significantly lower (below 150 ms in most cases) than that of the conventional cloud SFU.

layers, 360p base layer at 350 kbps and 720p enhancement layer at a total bitrate of 1 Mbps, so that peers receive guaranteed quality video at a pre-specified rate according to their terminal type. All peers with terminal type HD receive both layers and peers with SD terminal receive only one layer video. The VSP passes the terminal types of clients to the TEM unit of NSP, and TEM reserves bandwidth between end points within its edge cloud.

We picked the proportion of device types (received video resolution) for each peer according to the numbers projected for 2021 in the report [35], which estimates distribution of video terminal resolutions as 41%, 47% and 12% for SD, HD, and UHD, respectively. Since we only considered 720p and 360p video in our experiments, we set the probability that a peer has 360p terminal as 0.4 and 720p terminal as 0.6.

Test Videos: We used five 720p raw test videos, namely FourPeople (FP), Johnny, KristenAndSara (K&A), Vidyo3, Vidyo4 that are available in [36]. There is no global camera motion or large motion in these videos, which is typical of a real-time videoconferencing session. We encoded these videos using scalable VP9 with two spatial layers, where base layer is 360p encoded at 350 kbps and enhancement layer is 720p encoded at 1 Mbps (including the base layer). The PSNR values of the encoded videos at these bitrates are shown in Table 1 that indicate good quality. Since videos are delivered over paths with reserved bandwidth, the received videos have the same PSNR provided in Table 1.

Table 1
PSNR (dB) of scalable coded videos according to spatial layer.

Layer	FP	Johnny	K&A	Vidyo3	Vidyo4
720p	37.59	38.82	38.08	36.45	37.13
360p	33.30	35.92	33.37	31.61	33.19

6.4. Experimental results

We compare the performances of WebRTC services based on the proposed distributed edge-SFU vs. single edge-SFU (at a randomly

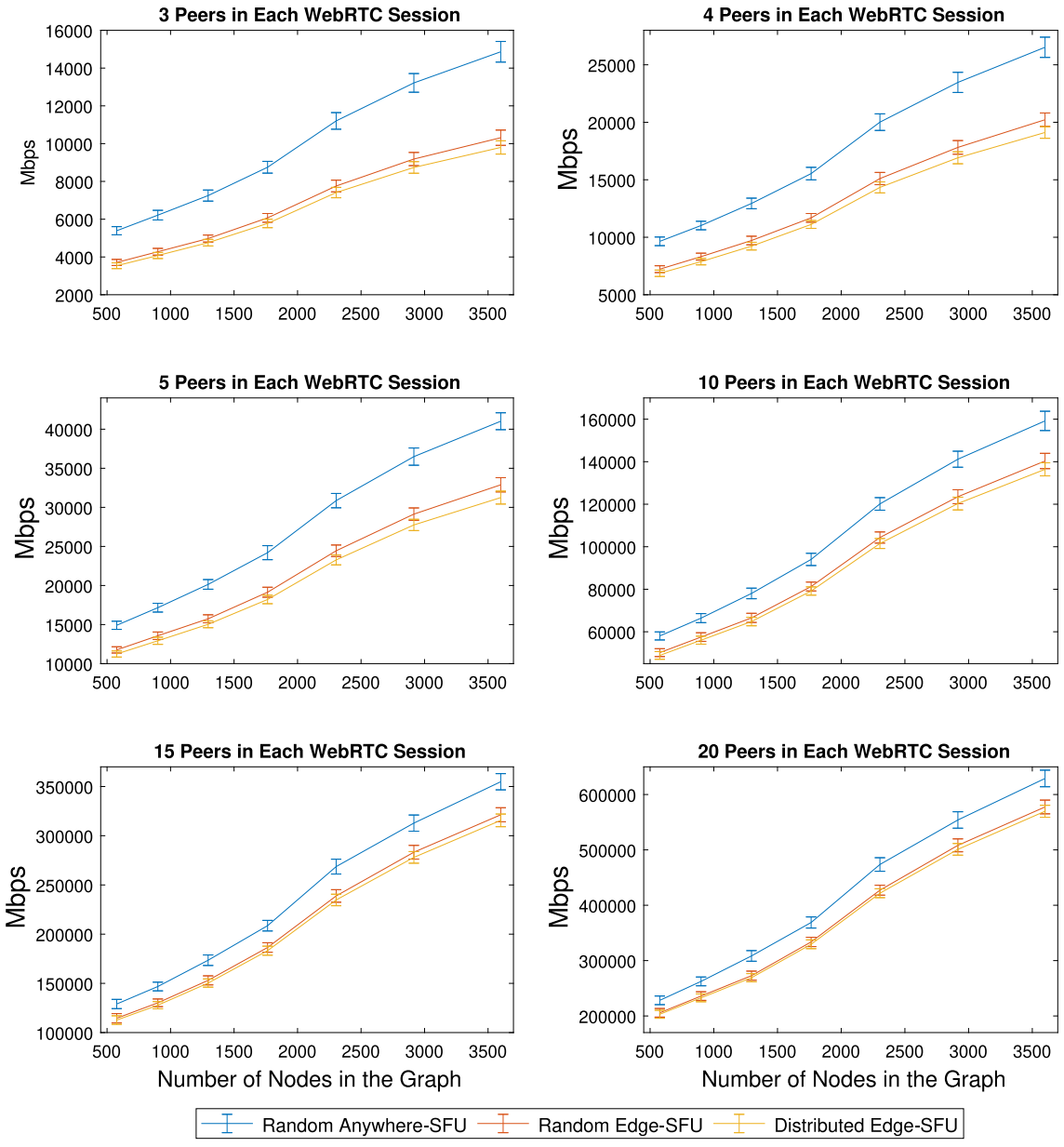


Fig. 9. Mean of E2E bandwidth consumption in 100 trials of 100 services. It can be seen that the mean bandwidth consumption with both random-edge and distributed edge-SFU schemes is lower than that with the conventional cloud SFU.

selected edge) and cloud SFU (placed randomly anywhere in the core network) in terms of two key performance indicators (KPI): (i) mean of maximum E2E delay of all WebRTC sessions that is a KPI relevant to end users, and (ii) total bandwidth consumption of all WebRTC sessions that is a KPI relevant to the NSP. Note that the video quality in all cases will be the same, since we assume the same bandwidth is reserved for all peers in all cases over edge network slices.

For each network size, we ran 100 tests each with 100 WebRTC sessions having 3, 4, 5, 10, 15 and 20 peers, respectively, for all cases of different SFU location scenarios. In each test, switch positions are randomly determined within their respective squares and links are randomly created according to Waxman probabilities (1) in the edge and core networks. The peers are connected to randomly chosen switches in randomly selected edge networks. In the cloud-SFU scenario, all peers are assigned to a single SFU that is connected to a randomly selected switch anywhere in the network. In the single edge-SFU scenario, all peers are assigned to an SFU located in the edge cloud of one of the peers in the session. Unlike the traditional single-SFU for a session scenario, in the distributed edge-SFU scenario, each peer is connected to the SFU in its own edge network with its sender PeerConnection object (sPCO) and other peers connect to this SFU with their receiver PeerConnection objects (rPCO) as explained in Section 4.2. In the special case, if all peers are in the same edge network, the session will be served by a single SFU.

The average and standard deviation of maximum delay and total bandwidth among all 100 sessions with 3, 4, 5, 10, 15 and 20 peers per session vs. network size are shown in Figs. 8 and 9, respectively. Delay results show the average over 100 trials of the maximum E2E delay among the 100 WebRTC sessions in each trial. Results show that, first of all, SFU deployment in edge networks significantly reduces network resource consumption and total E2E delay. Furthermore, delay and bandwidth results reveal that the proposed distributed edge-SFU WebRTC service offers value to both NSPs and end users. This is because the distributed edge-SFU architecture consumes less overall network resources and it provides much better worst case E2E delay performance, compared to the randomly placed cloud and single

edge-SFU architectures. These observations are consistent over all network sizes tested.

7. Conclusions

We propose an edge-integrated managed multi-party WebRTC service architecture to be provided by NSPs or NSP-authorized third-party service providers over resource-reserved edge network slices. The proposed architecture includes a modified signaling server and a NSP-WebRTC service manager that orchestrates the collaboration between the NSP and the WebRTC service.

The proposed multi-party WebRTC videoconference service involves following parties that need to work together for the end-to-end solution:

- NSP that provides the broadband access network service to its subscribers.
- Edge cloud service provider, which provides the cloud service for running the virtualized network control plane applications like SFU and WSM
- WebRTC service provider
- WebRTC peers, who will receive the WebRTC service

The first three parties can very well be the same entity, that is, an NSP can own the edge cloud and run its virtualized network applications on it, while providing WebRTC service to its customers, just like internet and IPTV services. In case WebRTC peers in a session receive service from different NSPs, there are two possibilities: the NSPs can share SFUs with each other, in accordance to peering agreements between them, or each NSP forces the peers to connect to its own SFU. Either way, the WebRTC service is capable of accommodating peers who are subscribed to different NSPs.

Our experimental results demonstrate that (i) in case all peers connect to a single SFU, placing the SFU at a network edge, where at least one of the peers reside lowers the mean of maximum delay compared to placing the SFU as a cloud service anywhere in the network, (ii) using multiple SFU servers, where each peer connects to the SFU in its own edge domain to send its outbound stream provides an even lower end-to-end delay.

Moreover, using multiple SFUs also results in the lowest overall total consumption of network resources for each WebRTC session. Hence, the proposed multiple SFU service is beneficial for both end users (lower delay) and for the NSP (lower use of network resources) compared to today's de facto single-SFU WebRTC communications. We note that there is no difference in received video quality for these different SFU configurations, since we allocate the same reserved (guaranteed) bandwidth in the edge networks in all cases.

Hosting special SFU switches at network edges under the control of edge datacenters operated by telecom operators may have other applications beyond the one discussed in this paper. Implementation of SFU switches at network edges under the control of edge datacenters operated by telecom operators for efficient implementation of WebRTC compliant multicast streaming services, such as browser-based IPTV services, will be investigated as future work.

Declaration of competing interest

We declare that this manuscript is original, has not been published before and is not currently being considered for publication elsewhere.

We confirm that the manuscript has been read and approved by all named authors and that there are no other persons who satisfied the criteria for authorship but are not listed. We further confirm that the order of authors listed in the manuscript has been approved by all of us.

We understand that the Corresponding Author is the sole contact for the Editorial process. He/she is responsible for communicating with the other authors about progress, submissions of revisions and final approval of proofs.

Data availability

No data was used for the research described in the article.

References

[1] L.L. Peterson, N. McKeown, M. Satyanarayanan, T. Anderson, G. Parulkar, O. Sunay, S. Katti, J. Rexford, A. Vahdat, Democratizing the network edge, Comput. Commun. Rev. 49 (2) (2019) 31–36.

[2] WebRTC source code, available online https://chromium.googlesource.com/external/webrtc/.

[3] VP9 video codec implementation. https://www.webmproject.org/vp9/.

[4] H. Alvestrand, Overview: Real time protocols for browser-based applications, 2017, draft-ietf-rtcweb-overview-19.

[5] G. Carlucci, L. De Cicco, S. Holmer, S. Mascolo, Analysis and design of the google congestion control for web real-time communication (webrtc), in: Proc. 7th Int. Conf. on Multimedia Systems, 2016.

[6] Scalable video coding (SVC) extension for webrtc W3C working draft, 2021, https://www.w3.org/TR/webrtc-svc/.

[7] H. Schwarz, D. Marpe, T. Wiegand, Overview of the scalable video coding extension of the H.264/AVC standard, IEEE Trans. Circuits Syst. Video Technol. 17 (9) (2007) 1103–1120.

[8] G.J. Sullivan, J.-R. Ohm, W.-J. Han, T. Wiegand, Overview of the high efficiency video coding (HEVC) standard, IEEE Trans. Circuits Syst. Video Technol. 22 (12) (2012) 1649–1668.

[9] A. Grange, P. de Rivaz, J. Hunt, VP9 bitstream and decoding process specification, 2016.

[10] Y. Chen, et al., An overview of core coding tools in the AV1 video codec, in: Picture Coding Symp, 2018.

[11] J. De Cock, A. Mavlankar, A. Moorthy, A. Aaron, A large-scale video codec comparison of x264, x265 and libvpx for practical VOD applications, in: SPIE Proceedings 9971, Applications of Digital Image Processing XXXIX, San Diego, CA, 2016.

[12] G. Bakar, R.A. Kirmizioglu, A.M. Tekalp, Motion-based rate adaptation in webrtc videoconferencing using scalable video coding, IEEE Trans. Multimedia 21 (2) (2019) 429–441.

[13] https://www.opennetworking.org/cord/.

[14] https://www.opennetworking.org/seba/;https://opennetworking.org/wp-content/uploads/2019/04/ONF-Reference-Design-SEBA-032919.pdf.

[15] F. Tashtarian, et al., A distributed delivery architecture for user generated content live streaming over HTTP, in: 2021 IEEE 46th Conf. on Local Computer Networks, LCN, 2021, pp. 162–169.

[16] F. Tashtarian, A. Bentaleb, A. Erfanian, H. Hellwagner, C. Timmerer, R. Zimmermann, HxL3: Optimized delivery architecture for HTTP low-latency live streaming, IEEE Trans. on Multimedia (2022) (Early access).

[17] A. Eleftheriadis, M.R. Civanlar, O. Shapiro, Multipoint videoconferencing with scalable video coding, J. Zhejiang Univ. Sci. A 7 (5) (2006) 696–705.

[18] E. Andre, L. Roux, A. Gouaillard, Comparative study of webrtc SFUs for video conferencing, in: Proc. Principles, Systems and Applications of IP Telecommunications (IPT Comm), 2018.

[19] S. Yoon, T. Na, H.-Y. Ryu, An implementation of web-RTC based audio/video conferencing system on virtualized cloud, in: IEEE Int. Conf. on Consumer Electronics, ICCE, 2016.

[20] K.-F. Ng, M.-Y. Ching, Y. Liu, T. Cai, L. Li, W. Chou, A P2P-MCU approach to multi-party video conference with webrtc, Int. J. Future Comput. Commun. 3 (5) (2014).

[21] A. Boubendir, E. Bertin, N. Simoni, Network as-a-service: the webrtc case: How SDN and NFV set a solid telco-OTT groundwork, in: Int. Conf. on Network of the Future, NOF, Montreal, Canada, 2015.

[22] S. Jero, V.K. Gurbani, R. Miller, B. Cilli, C. Payette, S. Sharma, Dynamic control of real-time communication (RTC) using SDN: A case study of a 5G end-to-end service, in: Proc. of IEEE/IFIP Network Operations and Management Symposium, NOMS, 2016.

[23] R.A. Kirmizioglu, B.C. Kaya, A.M. Tekalp, Multi-party webrtc videoconferencing using scalae VP9 video: From best-effort over-the-top to managed value-added services, in: IEEE Int. Conf. Multimedia and Expo, ICME, San Diego, CA, USA, 2018.

[24] R.A. Kirmizioglu, A.M. Tekalp, Multi-party webrtc services using delay and bandwidth aware SDN-assisted IP multicasting of scalable video over 5G networks, IEEE Trans. Multimedia 22 (4) (2020) 1005–1015.

[25] K.T. Bagci, K.E. Sahin, A.M. Tekalp, Compete or collaborate: Architectures for collaborative DASH video over future networks, IEEE Trans. Multimedia 19 (10) (2017) 2152–2165.

[26] S. Nacakli, A.M. Tekalp, Controlling P2P-CDN live streaming services at SDN-enabled multi-access edge datacenters, IEEE Trans. Multimedia 23 (2021) 3805–3816.

[27] M. Handley, SDP: Session description protocol, 2006, RFC 4566.

[28] J. Rosenberg, Interactive connectivity establishment (ICE): A protocol for network address translator (NAT) traversal for offer/answer protocols, 2010, RFC 5245.

[29] E.W. Dijkstra, A note on two problems in connexion with graphs, Numer. Math. 1, 269–271.
[30] Mininet Virtual Network, available online http://mininet.org/.
[31] Project Floodlight Open-Source SDN Controller, available online http://www.projectfloodlight.org/floodlight/.
[32] Janus Gateway, online https://github.com/meetecho/janus-gateway.
[33] E. Zegura, K. Calvert, S. Bhattacharjee, How to model an internetwork, in: Proc. IEEE Infocom, Vol. 2, 1996, pp. 594–602.
[34] G. Malkin, Traceroute using an IP option, 1993, RFC 1393.
[35] The Zettabyte Era: Trends and Analysis, Cisco Whitepaper, 2017, Available Online.
[36] Test videos https://media.xiph.org/video/derf/.